13.01 Data types

Primitive data types

In Chapter 12 we used variables to store values required by our algorithm. Look at Worked Example 12.01. The Identifier Table 12.02 lists two variable identifiers: `Miles` and `Km`. An identifier table should also show what sort of data (or data type) is going to be stored in each variable. The explanation shows that Miles will be a whole number, but that Km will be calculated using the formula Miles * 1.61. This will result in a number that may not be a whole number.

Primitive data types are those variables that can be defined simply by commands built into the programming language. Primitive data types are also known as atomic data types. In Computer Science a whole number is referred to as an INTEGER and a number with a decimal point is referred to as a REAL. Conditions are either TRUE or FALSE. These are logical values known as BOOLEAN. Sometimes we may want to store a single character; this is referred to as a CHAR.

A value that will always be a whole number should be defined to be of type INTEGER, such as when counting the iterations of a loop.



TIP

See Table 13.01 for a list of data types you should be familiar with.
See Chapter 1 (Sections 1.02 and 1.03) on how integers and characters are represented inside the computer. Chapter 16 (Section 16.03) covers the internal representation of real (single, double, float) numbers.

Further data types

If we want to store several characters; this is known as a string.

Note that there is a difference between the number 12 and the string "12".

The string data type is known as a structured type because it is essentially a sequence of characters. A special case is the empty string: a value of data type string, but with no characters stored in it.

When we write a date, such as 3 February 2018, we can also write this as a set of three numbers: 3/2/2018. Sometimes we might wish to calculate with dates, such as taking one date away from another to find out how many days, months and years are between these dates. To make it easier to do this, DATE has been designed as a data type. To see how different programming languages implement this data type, see Chapter 14 Section 14.03.

INTEGER	A signed whole number
REAL	A signed number with a decimal point
CHAR	A single character
STRING	A sequence of zero or more characters
BOOLEAN	The logical values TRUE and FALSE
DATE	A date consisting of day, month and year, sometimes including a time in hours, minutes and seconds

Table 13.01 Summary of pseudocode data types

TASK 13.01

Look at the identifier tables in Chapter 12 (Tables 12.06 and 12.09 to 12.12). Give the data type that is appropriate for each variable listed.

13.02 The record type

Sometimes variables of different data types are a logical group, such as data about a person (name, date of birth, height, number of siblings, whether they are a full-time student).

Name is a STRING; date of birth is a DATE; height is a REAL; number of siblings is an INTEGER; whether they are a full-time student is a BOOLEAN.

We can declare a record type to suit our purposes. The record type is known as a user-defined type, because the programmer can decide which variables (fields) to include as a record.



TIP

A record type is also known as a composite type.

In pseudocode a record type is declared as:

```
TYPE <TypeIdentifier>
    DECLARE <field identifier> : <data type>
    .
    .
ENDTYPE
```

We can now declare a variable of this record type:

```
DECLARE <variable identifier> : <record type>
```

And then access an individual field using the dot notation:

```
<variable identifier>.<field identifier>
```

Using the example above we can declare a Person record type:

```
TYPE PersonType
    Name : STRING
    DateOfBirth : DATE
    Height : REAL
    NumberOfSiblings : INTEGER
    IsFullTimeStudent : BOOLEAN
ENDTYPE
```

To declare a variable of this type we write:

```
DECLARE Person : PersonType
```

And now we can assign a value to a field of this Person record:

```
Person.Name ← "Fred"
Person.NumberOfSiblings ← 3
Person.IsFullTimeStudent ← TRUE
```

To output a field of a record:

```
OUTPUT Person.Name
```